서비스를 띄우는 것보다
계속 돌아가게 하는 일

클라우드 운영 · AX 솔루션 개발 지원자 이재환

앞으로는 운영 환경의 VM과 네트워크 구성부터
익히겠습니다. 장애가 난 뒤에 따라가던 것을 모니터링으로
먼저 알아채고, 반복되는 점검과 배포는 스크립트로
옮기겠습니다.

KT AIVLE School 수료
인턴 3회, 약 6개월
KT AIVLE School 빅프로젝트 우수상

구축

팀원들이 만든 웹 서비스와 YOLO 모델 서버를 AWS에 올렸습니다

수동 배포 → **GitHub Actions + OIDC 자동 배포**

Project 01, p.8, p.10

장애 추적

배포 직후 장애를 개발자 도구, curl, 로그로 끝까지 따라갑니다

배포 직후 로그인 실패 → **CORS 허용 목록 누락 찾아 수정**

Project 01, p.9

점검과 개선

띄운 뒤에도 안전하게 돌아가는지 다시 확인합니다

SSH 전체 공개 → **배포 중에만 러너 IP 허용 (개선안 설계)**

Project 01, p.10

AX 솔루션 개발
업무 분석과 화면 구현

전화 · 엑셀 고장 접수 → **요구사항정의서 · 화면설계서로 정리**
Project 02, p.15, p.19

공개 좌표로 어긋난 시설 위치 →
15개 시설 경계 GeoJSON 직접 구현
Project 01, p.13

GitHub github.com/jjaehani Tel 010-4654-0406 Email jjaehani@kakao.com

■ 기존 방식 ● 바꾼 결과

업무 분석부터 클라우드 운영까지 경험한 지원자 이재환입니다

클라우드 운영과 **AX 솔루션 개발** 지원 · KT AIVLE School 수료, 인턴 3회(약 6개월)



이재환

GitHub github.com/jjaehani
Tel 010-4654-0406
Email jjaehani@kakao.com

경력: 인턴 3회, 약 6개월

2023.07–08 **엔지니어스톡** 경영 부서 · 하계 인턴십 315시간
고장 접수 현행 분석, 3갈래 개선 설계, 요구사항 · 화면 설계

2023.01–02 **충남정보문화산업진흥원** 융합콘텐츠본부
DB 설계 보고서 · ERD, CSV 병합, 약 6,000만 건 이관(인턴 3명)

2022.09–12 **(주)아이와즈** 지능화기술연구팀
웹하드 침해 조사(게시물 약 140만 건), 학습 데이터 태깅 · 검수

주요 프로젝트

스마트 조선소 AI 안전관리 시스템 우수상 — Cloud, 디지털 트윈 · p.6
챗봇 기반 산업장비 고장 접수 솔루션 — 업무 분석, 설계 · p.14
인턴 하이라이트: DB 이관과 웹하드 조사 — p.21
그 밖의 프로젝트 4건 — p.22

학력, 교육, 병역

2024.02 **배재대학교** 사이버보안학과 졸업
2024.02 **DSC공유대학** 모빌리티SW/AI 융합전공
2026.09 **KT AIVLE School** 수료 (840시간)
2026.03 **육군** 만기전역 (2024.09–2026.03)

관련 수강: 웹서버구축및운영, 네트워크보안, LINUX계열보안, 운영체제보안, 데이터베이스보안, 데이터통신

자격, 어학

2023.09 **AICE Basic** 취득
2026.10 **토익스피킹** IM1

수상

2026.09 **KT AIVLE School** 빅프로젝트 우수상 (6인 팀)
2023.07 **DSC 기업애로기술 해결프로젝트 랩** 장려상 (5인 팀)
2022.04 **모빌리티 소부장 창업 아이디어 해커톤** 최우수상 (8인 팀)

한눈에 보는 핵심 역량 6가지

각 역량마다 실제로 해 본 일과 근거 페이지를 붙였습니다. 팀이 한 일과 제가 한 일은 본문에서 나눠 적었습니다.

역량	해 본 일	근거
클라우드 구축과 배포 AWS, Docker, GitHub Actions	웹 서비스(S3 · CloudFront, ALB · ECS Fargate, RDS)와 YOLO 모델 서버(EC2)를 AWS에 올리고, 수동 배포를 GitHub Actions + OIDC로 자동화	p.8–10
장애 추적과 우회 개발자 도구, curl, 로그	로그인 실패를 추적해 CORS 허용 목록 누락 수정, arm64 이미지는 amd64로 재빌드, GPU 제한은 CPU 인스턴스로 우회	p.9
보안과 비용 관리 IAM, MFA, 예산 알림	R&R 권한 매트릭스로 IAM 그룹 6개, 루트 MFA와 월 $10 예산 알림, OIDC로 장기 키 미저장, 자동 배포용 SSH 공개를 찾아 개선안 설계	p.10–11
업무 분석과 설계 문서 현행 분석, 요구사항, 화면	전화 · 문자 · 엑셀 고장 접수의 문제점 3가지를 정리해 3갈래로 개선 설계, 요구사항정의서(기능 9, 비기능 4)와 화면설계서 작성(㈜파이브웍스와 공동)	p.15–19
데이터 이관과 검증 MySQL, MariaDB	SKT 데이터 약 6,000만 건을 인턴 3명이 NAS MariaDB로 이관, 테이블별 건수와 속성 수 대조로 누락 확인	p.21
AI 활용과 직접 검증 Claude	Claude로 기존 코드 구조 파악, 범위는 직접 결정, 어긋난 지도는 15개 시설 경계를 직접 따서 수정	p.12–13

일하는 방식

범위부터 정하기
2인 · 약 2주에 맞춰 디지털 트윈을 2D 지도 중심으로 다시 정함 p.12

팀원 작업 살리기
구조도는 다시 그리고 구현은 팀원, 4명의 말뭉치는 버튼과 선택지로 살림 p.17

현업과 맞추기
자동 연동 제안을 대표 우려에 따라 담당자 확인 방식으로 절충 p.18

먼저 문서로 정하기
6,000만 건 이관 전에 작업 순서와 오류값 기준을 보고서로 정함 p.21

클라우드 운영과 AX 솔루션 개발에 맞춘 근거

클라우드 운영은 주요 업무 6가지별로, AX 솔루션 개발은 하는 일별로 제가 직접 맡은 역할을 붙였습니다.

클라우드 운영 주요 업무 6가지

퍼블릭 클라우드 인프라 운영	웹 서비스(S3 · CloudFront, ALB · ECS Fargate, ECR, RDS)와 모델 서버(EC2) 구성, Secrets Manager로 비밀값 주입, IAM 그룹 6개로 권한 분리	p.8
클라우드 VM 운영 및 유지보수	EC2 모델 서버: GPU 인스턴스(g4dn.xlarge)가 무료 요금제 제한으로 막히자 CPU 인스턴스(m7i-flex.large)로 우회. 인턴 때는 kt cloud 원격 서버가 멈추자 다른 서버에 분석 프로그램을 재설치해 작업 재개	p.9, p.21
클라우드 네트워크 모니터링 및 운영	CloudFront 도메인 하나로 프론트와 API(/api/* → ALB) 연결, CORS 누락 추적, DB 외부 접근 차단, 자동 배포용으로 열어 둔 SSH 전체 공개 발견	p.9–10
운영 환경 개선 및 자동화	수동 배포를 GitHub Actions + OIDC로 자동화(장기 액세스 키 없음, 이미지 태그는 커밋 SHA), 모델 서버 SSH 개선안 설계	p.10–11
AI 인프라 환경 운영	YOLO 모델 서버: 카메라 송출 → MediaMTX → 판정 · 비식별화 → RTSP/HLS 전 구간 동작 확인 (CPU 인스턴스, 약 3FPS 한계)	p.8–9
클라우드 환경 모니터링 및 운영	장애 후 개발자 도구 · curl · 로그로 원인 추적, 월 $10 예산 알림 다음 목표 모니터링으로 먼저 알아채기	p.9, p.11

함께 보는 경험

퍼블릭 클라우드 사용 경험 AWS로 웹 서비스와 모델 서버 구축 · 배포 · 장애 대응 p.8–11

운영 자동화 및 스크립트 활용 GitHub Actions + OIDC 자동 배포, 다음에는 반복 점검도 스크립트로 p.10–11

전공 기반 사이버보안학과에서 웹서버 구축 · 운영, Linux · 운영체제 보안, 네트워크 보안 수강 p.2

AX 솔루션 개발 업무 시스템 · AI

웹 업무 시스템과 데이터 연계

화면 입력이 MySQL에 바로 저장되도록 PHP · Apache 구조를 다시 그리고, Issue 테이블에 정형 분류와 고객 원문을 나눠 저장

p.16–17

업무 분석과 SI 산출물

전화 · 문자 · 엑셀 고장 접수의 문제점 3가지를 정리해 3갈래로 개선 설계, 요구사항정의서(기능 9, 비기능 4) · 화면설계서로 옮김

p.15–19

챗봇 시나리오와 AI 개선안

카카오 i 오픈빌더로 고장 접수 챗봇 시나리오 구성(스킬 서버 연동은 팀원), LLM 증상 매핑은 구현하지 않은 개선안

p.19, p.20

AI 활용과 직접 검증

Claude로 코드 구조 파악, 범위는 직접 결정, 어긋난 지도는 15개 시설 경계를 직접 따서 수정

p.12–13

스킬은 실제로 쓴 프로젝트와 함께 적었습니다

주력 분야와 사용해 본 분야를 나누고, 각 기술을 쓴 프로젝트 페이지를 붙였습니다.

● 주력 분야

클라우드 / 운영	AWS(IAM, EC2, ECR, RDS, S3, CloudFront, Secrets Manager), Docker, GitHub Actions(OIDC), 장애 추적(개발자 도구, curl, 로그), 모델 서버 영상 흐름(MediaMTX), 예산 알림	Project 01, p.8–11
AX 기획 / 업무 분석	현행 업무 분석, 개선업무설계, 요구사항정의서, 화면설계서, 디자인씽킹(페르소나 · 사용자 인터뷰), Figma, 카카오 i 오픈빌더	Project 02 p.15–19, p.22
데이터 분석 / AI 모델	Python, Selenium 크롤러, CSV 병합 코드, YOLOv4-tiny(Darknet, Colab) · YOLOv5x 학습, labelImg 라벨링, json → YOLO txt 라벨 변환	인턴 p.21, 그 밖의 프로젝트 p.22

○ 사용해 본 분야

DB / 백엔드	MySQL · MariaDB, DataGrip, phpMyAdmin, ERD Cloud, ERD 설계 · 정규화, PHP · Apache 프로토타입 구조 설계	Project 02 p.17–19, 인턴 p.21
AI 활용 / 분류 체계	Claude로 코드 구조 파악 후 실행 화면으로 검증, 고장 증상 말뭉치 기반 분류 체계	Project 01 p.12, Project 02 p.16
웹 화면 / 지도	React, Kakao Map API, GeoJSON 폴리곤, 3D 설비 상세 화면	Project 01, p.13
보안 / 기타	Burp Suite(GET 파라미터로 페이지 이동), x32dbg로 서버 도메인 · IP 추적, kt cloud 원격 서버, Ubuntu	인턴 p.21

KT AIVLE School 빅프로젝트

스마트 조선소 AI 안전관리 시스템

클라우드와 디지털 트윈을 맡았고
팀이 우수상을 받은 AI 안전관리 플랫폼

기간	팀	내 역할
2026.06–08	6인 (디지털 트윈 파트 2인)	클라우드, 디지털 트윈

내가 맡은 일

- **AWS 인프라 설계 · 구축 · 배포** p.8–11

구성	AWS	포인트
프론트	S3 + CloudFront	CORS 장애 추적 (p.9)
백엔드	ECS Fargate (ALB 경유)	**GitHub Actions + OIDC 자동 배포**
DB	RDS, 외부 접근 차단	비밀번호는 Secrets Manager
모델 서버	EC2 CPU 인스턴스*	**GitHub Actions 자동 배포**

* 설계는 GPU 인스턴스였으나 무료 요금제 제한으로 CPU 인스턴스에 올림

- **공간형 디지털 트윈 지도 · 3D 설비 상세 뷰** p.12–13
  15개 시설 경계 GeoJSON 폴리곤, React · Kakao Map API

우수상
KT AIVLE School 빅프로젝트

IAM 그룹 6개
팀원 인터뷰로 만든 R&R 권한 매트릭스

2인 · 약 2주
디지털 트윈 범위 재설계 후 완성

팀 스택 Spring Boot, MySQL, RAG + LLM, YOLO



직접 구현 SHIP-SAY 공간형 디지털 트윈 지도 화면

조선소의 고위험 작업을 미리 찾고 알리는 4가지 기능

6인 팀에서 **클라우드 인프라**와 **디지털 트윈(공간형 지도 구현 · 범위 재설계)**을 맡았습니다.

기존 방식 → 바꾸려는 방식

순찰과 수기 확인 → 사전 예측과 경고

배경

고위험·동시 작업이 늘어 실시간 안전관리가 필요

배경

순찰과 수기 확인 중심이라 위험 조기 발견에 한계

배경

숙련 외국인력 비자 4년 새 약 50배*, 안전교육과 소통 지원 필요

제약

공석이 된 디지털 트윈 파트를 **2명이 약 2주** 안에 개발

담당 내가 맡은 기능 **연동** 팀원 개발 기능 클라우드는 팀원들이 만든 웹 서비스와 YOLO 모델 서버를 AWS에 올리는 일로 맡았습니다(p.8).

연동

AI 작업허가서 검토

작업 위험 분석, 안전조치 제안

연동

다국어 TBM 안전 브리핑

안전수칙 생성, 번역, 음성 안내

담당

디지털 트윈 지도 · 사전 경고

작업자 위치·위험구역 표시, 진입 경고 (예측 모델은 팀원)

연동

마스킹 CCTV 모니터링

비식별 기반 보호구, 위험 감지

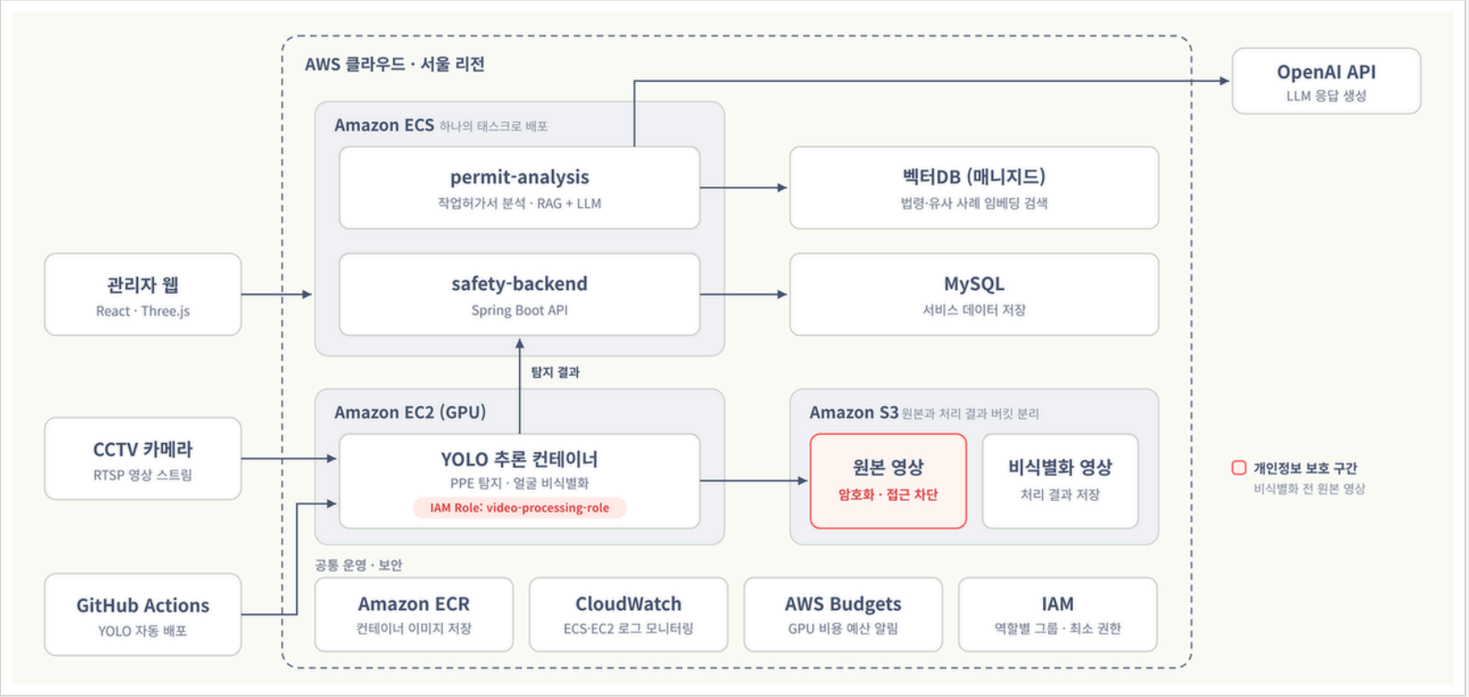
* 조선업 숙련 외국인력(E-7-3) 비자 발급 건수: 2021년 264건 → 2025년 1만 3,297건 (법무부 정보공개청구 자료, 스마트투데이 2026.03 보도)

팀원들이 만든 웹 서비스와 YOLO 모델 서버를 AWS에 올렸습니다

팀원 인터뷰로 실행 환경을 먼저 정리하고, **3주 일정**에 맞춰 계정 · 권한 → 웹 인프라 → 모델 서버 → CI/CD 순으로 구축했습니다.

실제로 올린 구성 (서울 리전)

진입	**CloudFront** 도메인 하나로 프론트(S3)와 API(/api/* → ALB → ECS Fargate)를 함께 제공
백엔드	**ALB → ECS Fargate**(0.5 vCPU, 1GB), 이미지는 ECR, GitHub Actions + OIDC 자동 배포
DB	**RDS MySQL** 외부 접근 차단, 비밀번호는 **Secrets Manager**, 업로드 파일은 비공개 S3
모델 서버	**EC2 CPU 인스턴스**, GitHub Actions 자동 배포 (p.9, p.10)
계정 · 권한	계정 1개에 **R&R 권한 매트릭스**로 IAM 그룹 6개, 루트 MFA(패스키), 월 $10 GPU 예산 알림



직접 설계 설계 기준 아키텍처. 실제 모델 서버는 CPU 인스턴스로 운영했고, 원본 영상 버킷과 보안그룹 계층은 서울 서비스에 적용하지 못했습니다(p.11).

실제 진행 순서 (클라우드 3주 압축 일정)

시작
팀원 인터뷰: 프론트는 정적 파일 → S3 · CloudFront, 백엔드는 Spring Boot · MySQL 8.4 → 컨테이너 · RDS

7/16–21
계정 · IAM · 권한, 네트워크 · 보안그룹 · S3

7/22–24
서울 리전 웹 인프라, ECR 푸시 → 태스크 정의 → ECS 서비스 배포

7/27
웹 서비스 배포, **로그인 장애 해결**

7/27–28
모델 서버 CPU 배포, 스트림 검증

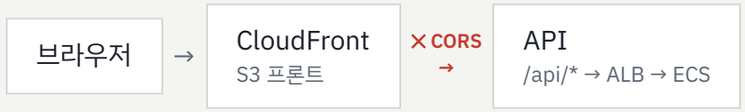
7/29–30
웹 · 모델 서버 **CI/CD**, 8/26 최종 발표

배포 직후 장애를 개발자 도구, curl, 로그로 끝까지 따라갔습니다

기능이 완성돼 있어도 배포 직후 로그인이 실패하거나 모델 서버가 뜨지 않으면 시연을 할 수 없었습니다.

장애 1

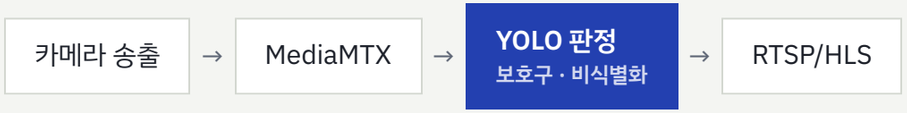
배포 직후 로그인이 실패했습니다



증상	배포 직후 로그인 실패
추적	개발자 도구와 curl로 요청을 따라감
원인	CORS 허용 목록에서 **CloudFront 도메인이 빠져 있음**
조치	**백엔드 팀원과 허용 목록을 고쳐 재배포**
함께	Apple Silicon에서 만든 arm64 이미지를 amd64로 다시 빌드

장애 2

GPU 인스턴스가 막혀 모델 서버가 뜨지 않았습니다



증상	GPU 인스턴스(g4dn.xlarge)가 무료 요금제 제한으로 막힘
조치	**CPU 인스턴스(m7i-flex.large)로 우회해 전체 흐름부터 검증**
확인	카메라 송출부터 판정 · 비식별화, 스트림 출력까지 끝까지 확인

성과는 GPU 없이 송출 → 판정 · 비식별화 → 재송출 전 구간이 돌아가는 것을 확인한 것입니다. 처리 속도는 로그 기준 약 3FPS로, 실시간 감시에는 부족해 GPU 인스턴스가 필요한 한계가 남았습니다.

수동 배포를 자동화하고, 직후 점검에서 SSH 전체 공개를 찾았습니다

서비스를 띄우는 것보다 **띄운 뒤 안전하게 계속 돌아가게 하는 일**에 관심이 생긴 계기입니다.

배포 방식

웹 서비스: 수동 배포에서 단계를 빠뜨리고 이미지 버전이 헷갈려 GitHub Actions + OIDC로 자동화

main 푸시 → OIDC로 역할 받기 → 이미지 빌드 · ECR (태그는 커밋 SHA) → **ECS 배포** (약 5분)

프론트 빌드 → S3 업로드 → **CloudFront 무효화** (약 30초)

모델 서버: 코드 변경이 잦아 GitHub Actions 자동 배포

코드 변경 → GitHub Actions → SSH 접속 → **컨테이너 재빌드**

실행 성공까지 확인 · 반복 배포 실수 감소, 감수한 점은 초기 설정 시간

동작 확인 직후 점검

발견: 자동 배포를 위해 제가 열어 둔 SSH

SSH가 전체 공개로 열려 있음

설계한 개선안 · 기간 안에 적용하지 못함

배포하는 동안에만 GitHub Actions 러너 IP를 엽니다

배포 시작 → 러너 IP만 허용 → 배포 → 배포가 끝나면 닫기

함께 적용한 권한 · 보안 · 비용 관리

권한

인터뷰로 만든 R&R 권한 매트릭스로 IAM 그룹 6개를 나누고, 배포 전용 역할에는 배포 권한만 줬습니다. 루트는 MFA(패스키)를 걸었지만 IAM 사용자 MFA는 적용하지 못했습니다.

보안

OIDC로 장기 액세스 키를 저장하지 않고 신뢰 정책은 main 브랜치 요청만 허용했으며, DB 비밀번호는 Secrets Manager에서 태스크 시작 시 주입했습니다. 보안그룹 계층과 원본 영상 버킷은 설계 · 구축했지만 서울 서비스에는 적용하지 못했습니다(p.11).

비용

월 $10 GPU 예산 알림으로 비용을 관리했습니다.

6인 팀이 감당할 수 있는 운영 범위로 설계했습니다

결정마다 **얻은 것과 감수한 점**을 함께 정리하고, 다음 단계까지 잡아 두었습니다.

결정	얻은 것	감수한 점
AWS 계정 1개, 태그로 구분	6인 팀의 관리 단순화	환경 간 격리가 약함
기본 VPC 재사용, 서브넷 최소화	기간 안에 빠르게 구축	네트워크 분리가 약함
DB는 매니지드 서비스(RDS)	운영 인력 부담 감소	비용 증가
수동 배포 → GitHub Actions 자동화	반복 배포 실수 감소	초기 설정 시간

핵심 팀이 감당할 수 있는 범위로, **관리할 대상을 줄이는 쪽**을 택했습니다.

배운 점 퍼블릭 · 프라이빗 서브넷, 보안그룹 계층, 원본 영상 버킷을 **us-east-1**에 만들어 서울 리전 서비스에 쓰지 못했습니다. 리소스를 만들기 전에 **리전과 계정 기준을 팀과 먼저 맞춥니다.**

다음에 한다면 · 구현하지 않은 개선 제안

- 개발 및 운영 환경을 계정 단위로 분리
- private 서브넷으로 네트워크 분리
- Terraform으로 인프라를 코드로 관리
- CloudWatch 알람을 메신저로 연결
- S3/DB 백업 정책 수립
- 테스트를 통과했을 때만 배포
- 배포 실패 시 이전 태스크 정의로 되돌리기
- IAM 사용자에도 MFA 적용

빅프로젝트에서는
장애가 난 **뒤에** 개발자 도구와 curl, 로그로 원인을 따라갔습니다.

→ **앞으로는** 계획
운영 환경의 VM과 네트워크 구성 익히기 → 모니터링으로 이상을 먼저 알아채기 → 반복 점검과 배포를 스크립트로

공석이 된 디지털 트윈을 넘겨받아 범위를 다시 정했습니다

디지털 트윈 담당자가 그만둬 제가 이어받았습니다. 개발 인원은 2명, 기간은 약 2주였습니다.

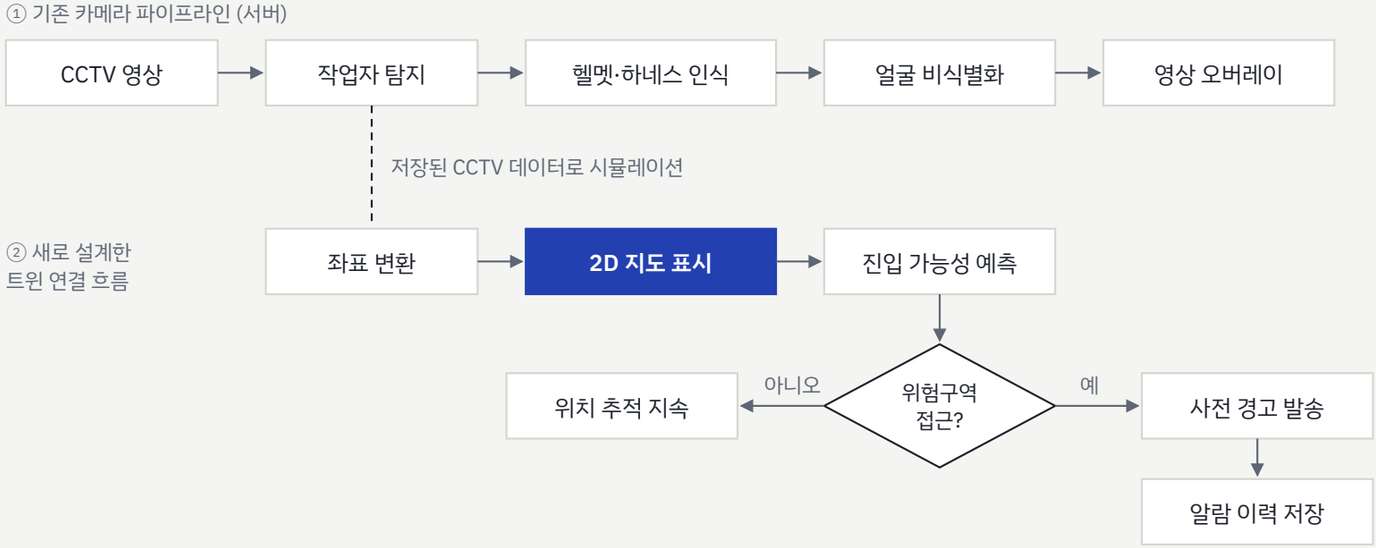
범위 정리: “트윈이냐 관제냐” 논쟁

CCTV와 센서 기반 **2D 공간형 디지털 트윈**으로 정의를 문서화했습니다.

야드 전체 3D	→ **2D/2.5D 지도**
로봇팔 6축 애니메이션	→ **제외**
설비 상세	→ **3D 뷰로 한정**
실제 장비 제어	→ **범위 제외**
카메라 탐지	→ **트윈 연결** 새로 설계
범위와 정의	→ **문서로 확정**

MVP 시나리오: 위험구역 진입 사전 경고

카메라 결과가 영상 오버레이로만 나와서, 트윈과 연결하는 흐름을 설계했습니다.



직접 구현 2D 지도 표시 지도 표시 · 사전 경고: 검증 데이터로 동작 점선: 실시간 카메라 연결은 저장된 CCTV 데이터로 시뮬레이션

AI 활용과 직접 검증

과제

처음 보는 코드를 짧은 기간에 파악해, 끝낼 수 있는 범위로 다시 만들어야 했습니다.

AI에게 맡긴 것

Claude에 코드를 파일 단위로 보여 주며 모듈별 역할과 연결 관계를 정리하게 했고, 3D 유지안과 2D 전환안의 작업량 비교 초안을 받았습니다.

직접 정한 것

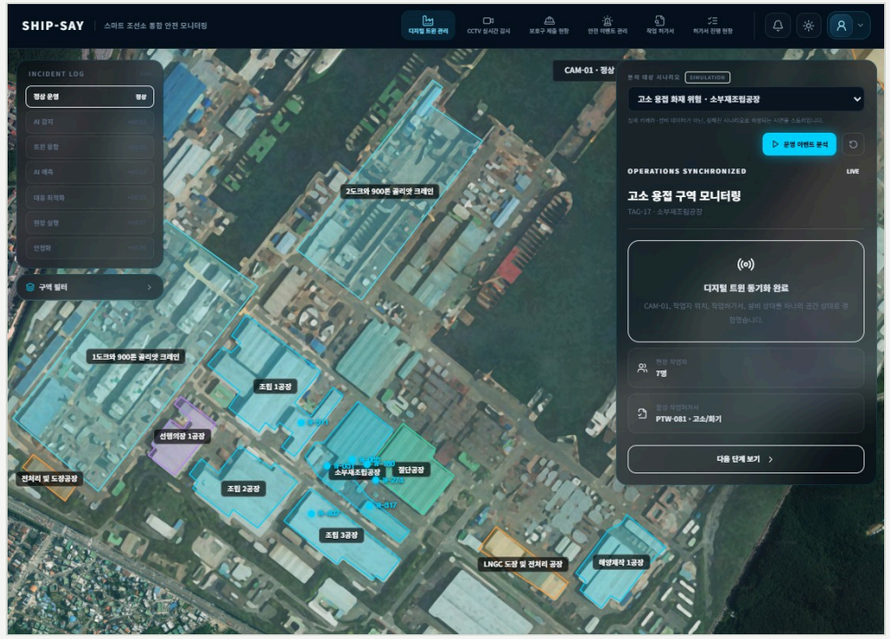
범위는 제가 정했습니다. 야드 전체 3D는 2D 지도로 바꾸고 로봇팔 애니메이션은 뺐으며, 3D는 설비 상세 화면에만 남겼습니다.

직접 확인한 것

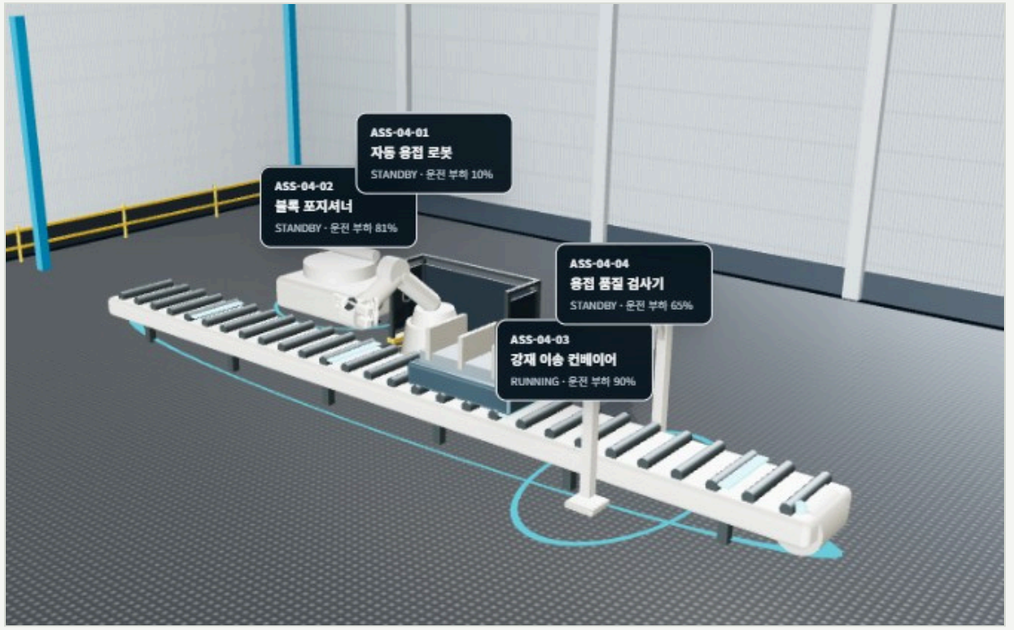
AI가 정리한 연결 관계는 실행 화면과 로그로 확인했습니다. 화면에서 찾은 문제는 p.13에 정리했습니다.

위성 지도에서 15개 시설 경계를 직접 따서 디지털 트윈 지도를 구현

카카오맵 위성 지도에 한화오션 거제사업장 15개 시설을 GeoJSON 폴리곤으로 표시했습니다. 인턴 때 카카오 챗봇을 다뤄 카카오 플랫폼에 익숙해서 **Kakao Map API**를 골랐고, 화면은 React로 만들었습니다.



직접 구현 공간형 DT 지도: 위험구역 진입 시 폴리곤 깜빡임, 알림 배너, 구역 상세 패널



직접 구현 3D 설비 상세 화면: 부품 진단, 센서 패널, 작업자 위험도, 사고 대응 콘솔

15개
위성 지도에서 직접 따서 GeoJSON 폴리곤으로 표시한 시설

2D + 3D
야드는 2D 지도, 3D는 설비 상세 화면에만

SIMULATION
시연용 수치는 화면에 따로 표기

Trouble Shooting: AI 안내대로 띄운 지도를 실행 화면으로 검증

문제	Claude 안내대로 띄운 지도가 오른쪽 일부만 확대돼 보임	**조치**	초기 확대 수준을 낮추고 중심 좌표를 잡아 전체가 보이게 함
문제	시설 위치가 실제와 어긋남 (보안구역이라 공개 좌표가 부정확)	**조치**	위성 지도에서 15개 시설 경계를 하나씩 직접 따서 이름과 함께 GeoJSON 폴리곤으로 표시

시나리오(화재 · 가스누출 · 크레인) 화면에는 팀원이 학습한 MLP 예측 결과를 연결하고, 시연용 수치는 SIMULATION으로 표기해 실제 데이터와 구분하고, 근거가 없는 성능 수치는 '목표'로 적었습니다.

DSC 기업애로기술 해결프로젝트 랩(장려상)에서 인턴으로 이어진 과제

챗봇 기반 산업장비 고장 접수 솔루션

전화로 받던 산업장비 고장 접수를 챗봇 접수 흐름과 설계 문서로 옮겼습니다

기간	2023.06–08 (프로젝트 → 인턴)
담당	업무 분석, 설계 문서, 증상 말뭉치
고객사	엔지니어스톡 (산업장비 고장 접수)
협업	챗봇 개발사 ㈜파이브웍스 (인턴 단계)

현행 분석
문제점 3가지
수리 · 영업관리 · 구매 · 출하관리 흐름도
p.15

데이터 구조화
3 / 10개
제품군 중 PCB · DRV · HMI 증상 말뭉치 담당
p.16

개선 설계
3갈래
수리 · 구매에 챗봇 자동화, 내역조회 신설
p.18

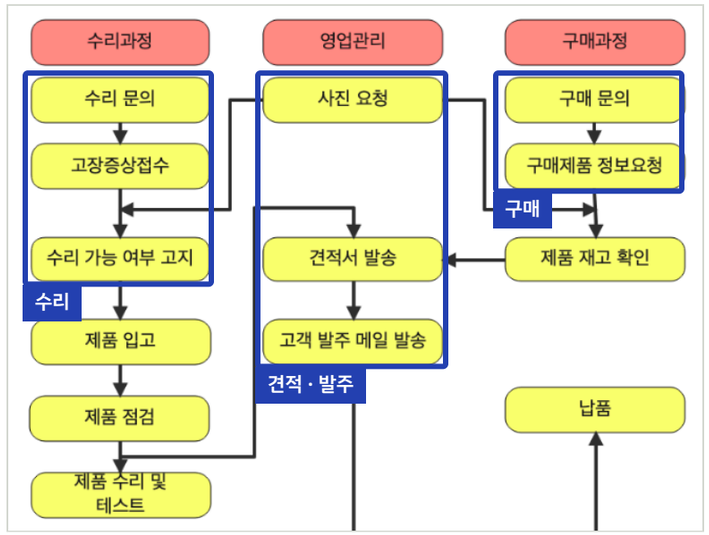
요구사항
기능 9개
비기능 4개, 요구사항정의서 · 화면설계서
p.19

프로젝트 2023.06–07 · 5인 · 장려상: 3개 제품군 말뭉치, Figma 카카오톡 UI 프로토타입, PHP 구조 제안 · 구조도 (p.17)

인턴 2023.07–08 · 경영 부서: 현행업무분석서, 개선업무설계서, 요구사항정의서, 화면설계서, 오픈빌더 시나리오

기존 업무(AS-IS) 흐름도와 챗봇 자동화 구간

현행업무분석서에서 정리한 문제점 3가지와, 개선 설계에서 바꾼 방법



남색 테두리: 현행업무분석서에서 표시한 챗봇 자동화 구간

1 증상을 전화로 하나하나 설명
장비가 멈춘 급한 상황에서도 오래 걸림

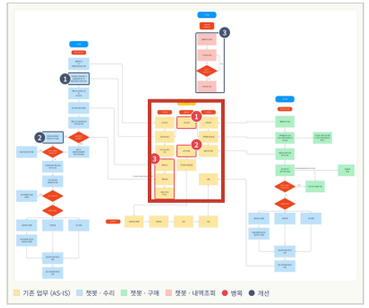
1 챗봇으로 수리 요청
채팅과 사진 첨부로 접수

2 전화 · 문자 · 카톡 · Excel로 나뉨
연락처, 사진, 상담, 입고를 따로 관리

2 수리 · 구매 단계에 챗봇 자동화
기존 흐름은 두고 수동 입력을 줄임

3 손으로 정리해야 현황 파악
고객 · 접수 · 고장 정보를 수기로 정리

3 내역조회 신설
관리자 페이지와 챗봇 연동, 고객이 직접 조회



전체 흐름도
빨간 테두리가 위에서 확대한 기존 업무 영역입니다.
둘레의 챗봇 흐름(수리 · 구매 · 내역조회)은 p.18

전화, 문자, 엑셀로 나뉜 고장 접수를 흐름도로 따라가 문제점 3가지를 정리했습니다

인턴 단계에서 **수리, 영업관리, 구매, 출하관리** 흐름을 따라가며 흐름도를 그리고 현행업무분석서를 썼습니다.



현행업무분석서의 문제점 3가지

문제 1

장비가 멈춘 급한 상황에서도 고객이 고장 증상을 전화로 하나하나 설명해야 해서 시간이 오래 걸림

문제 2

전화로 연락처를 남기고, 문자나 카톡으로 사진을 보내고, 다시 전화로 상담한 뒤 입고해 수리하는 과정을 Excel로 관리함

문제 3

고객 정보, 접수 정보, 고장 정보를 손으로 정리해야 현황을 알 수 있음

흐름도에 표시한 챗봇 자동화 구간

수리 문의 → 수리 가능 여부 안내

↓

사진 요청 → 고객 발주 메일

↓

구매 문의 → 제품 정보 요청

흐름도 확대는 p.14

다음 단계 문제점 3가지를 바탕으로 개선업무설계서에서 업무를 **수리 · 구매 · 내역조회 3갈래**로 다시 나눴습니다(p.18).

근거: 현행업무분석서 (엔지니어스톡 인턴, 2023.07–08)

10개 제품군의 고장 증상을 제품군별 대분류 → 세부 증상 체계로

10개 제품군의 고장 증상 말뭉치를 4명이 나눠 썼고, 저는 **PCB, DRV, HMI**를 맡았습니다.

고장 유형 분류 체계 내 담당 제품군

PCB

- 통신
- 출력
- 전원
- 동작
- 체크
- 작동

DRV

- 알람
- 전류
- 전원
- 회로
- 오버

HMI

- 터치
- 화면
- 전원
- 부팅

PCB 6개, DRV 5개, HMI 4개 대분류 아래에 세부 증상을 정리했습니다. 회사는 고장 증상을 제품 카테고리별로 분류해 달라고 요청했고, 대상은 접수가 많은 10개 제품군(HMI, PCB, 인버터, 컨트롤러, 서보드라이브 등)이었습니다.

방법 수리내역과 제품 종류 엑셀을 보고 **접수가 많은 10개 제품군**을 고른 뒤, 4명이 나눠 **대분류와 세부 증상**을 썼습니다.

DB 설계: Issue 테이블

`issue_category`	정형 분류
`symptom_description`	고객 원문

분류는 제품군 버튼과 증상 체크로 받고, 고객이 직접 쓰는 보조 텍스트는 원문 그대로 저장합니다(p.17).
전체 ERD는 p.19

핵심

**고객 표현은 원문 그대로 남기고,
관리는 정형 분류로 합니다.**

완성 기한 2주 전, 저장이 안 되는 구조를 바꾸고 팀원들의 작업은 살렸습니다

2023 DSC 기업애로기술 해결프로젝트 랩. 목표는 최종 발표 때 **실제로 접수되고 데이터가 쌓이는 프로토타입**이었습니다.

기존: 팀원이 만든 Django 버전

Django 백엔드
로그인 · 자연어 처리

↓

공용 MySQL 연동에서 막힘

↓

완성 기한(7/7)까지 2주도 안 남음

접수 내용이 DB에 남지 않음

판단

PHP와 Apache로 **화면 입력을 바로 MySQL에 저장**하는 방식을 제안했습니다. 보안과 확장성이 떨어진다는 것은 알았지만, 기한 안에 데이터가 실제로 쌓이는 구조가 먼저라고 판단했습니다.

구조도는 **제가 다시 그렸고**, 팀원이 그 구조도대로 구현해 6월 말 약 70%까지 진행했습니다.

→

바꾼 구조: 최종 발표 버전

로그인 · 회원가입(개인, 기업)
버튼형 고장 접수

↓

PHP · Apache
내 구조도, 팀원 구현

↓

MySQL에 저장

7/14 최종 발표, 장려상

팀원들의 작업을 살린 곳

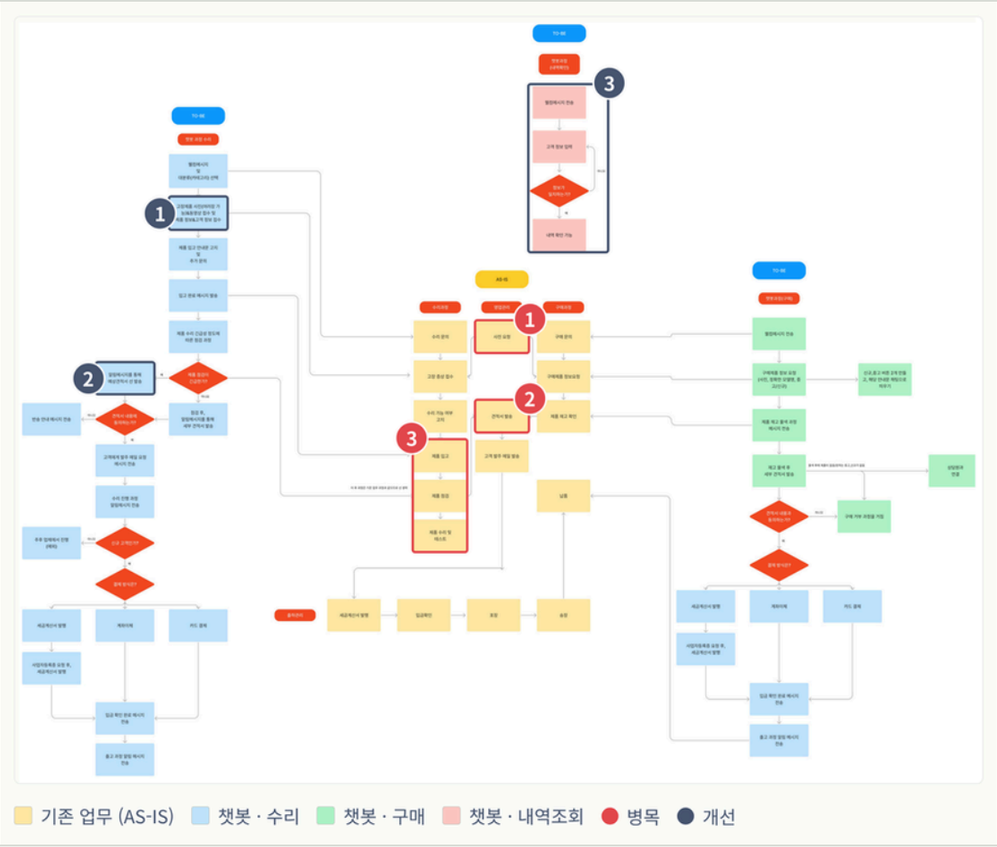
4명이 쓴 10개 제품군 말뭉치	**→ 제품군 버튼과 증상 체크 선택지의 기준**
고객이 직접 쓴 원문	**→ 버리지 않고 Issue 테이블에 따로 저장**
텍스트만 입력하는 자연어 접수	→ 팀원과 상의해 제외

> **배운 점** 더 좋은 기술보다 **기한 안에 팀이 만든 결과물이 이어지는 방법**을 고르는 것도 판단입니다.

초반 범위 점검: 챗봇까지 우리가 만드는지 ㈜파이브웍스에 먼저 확인하자고 제안했고, 실제 카카오톡 챗봇은 파이브웍스와 인턴 인원이 맡기로 정리됐습니다. 우리 팀은 같은 접수 흐름을 **카카오톡 UI 프로토타입(Figma, 제가 설계)**으로 먼저 보여 주고 DB를 설계했습니다.

문제점 3가지를 바탕으로 업무를 수리 · 구매 · 내역조회 3갈래로 다시 설계했습니다

개선업무설계서(인턴 단계)에서 기존 흐름은 살리고, 챗봇이 맡을 부분과 새로 만들 흐름을 나눴습니다.



가운데 노랑: 기존 업무 · 왼쪽 파랑: 챗봇 수리 · 오른쪽 초록: 챗봇 구매 · 위 분홍: 내역조회(신설)

갈래	설계 내용	관련 기능 요구사항
수리	기존 수리 흐름의 각 단계에 챗봇 자동화를 붙여 수동 입력을 줄임	수리 요청, 사진 첨부, 진행 상황 알림
구매	구매 문의부터 제품 정보 요청까지를 챗봇으로 받음	구매 요청
내역조회 신설	관리자 페이지와 챗봇을 연동해 고객이 직접 내역과 배송을 조회	내역 확인 요청

자동화 범위는 현업과 맞췄습니다

제안	챗봇으로 받은 접수가 수리내역에 바로 들어가는 자동 연동
우려	고객이 잘못 입력하면 수리내역 자체가 틀어짐 (대표 의견)
결정	**최종 입력은 업무 담당자가 확인하고 채움**

자동화 범위는 기술적으로 가능한지보다, 현장에서 틀렸을 때 누가 고치는지를 먼저 따져 정했습니다.

요구사항정의서와 화면설계서로 옮기고, 챗봇 시나리오를 구성했습니다

인턴 단계(2023.07–08, 경영 부서)에서 대표 아래 업무 분석과 설계 문서를 맡았고, ㈜파이브웍스와 맞춰 가며 작성했습니다.

내 역할

01 현행 업무 분석

4개 흐름의 흐름도, 챗봇 자동화 구간 표시, 문제점 3가지 정리 (현행업무분석서)

02 개선 설계

수리 · 구매 · 내역조회 3갈래로 업무를 다시 나눔 (개선업무설계서)

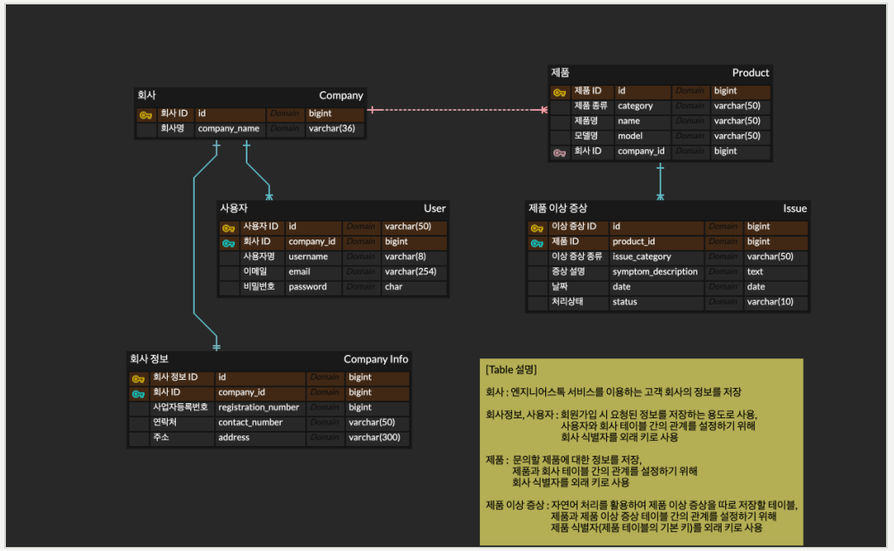
03 요구사항 · 화면 설계

요구사항정의서와 화면설계서 작성, ㈜파이브웍스와 공동

04 챗봇 시나리오

카카오 i 오픈빌더로 고장 접수 대화 시나리오를 직접 구성 (스킬 서버 · API 연동은 팀원)

ERD 5개 테이블 (DSC 프로젝트 때 팀 공동 설계)



Company, User, CompanyInfo, Product, Issue. 고객 원문은 Issue 테이블에 따로 저장합니다 (p.16).

인턴 산출물 4종

- **요구사항정의서** — 기능 9 · 비기능 4
- **화면설계서**
- **현행업무분석서**
- **개선업무설계서**

기능 9 챗봇 호출 버튼, 카카오 챗봇 실행, 사진 첨부, 채팅 송수신, 구매 요청, 수리 요청, 내역 확인 요청, 진행 상황 알림, 주제를 벗어난 질문 예외 처리

비기능 4 서버 사양, 서버 OS(Ubuntu 20.04 LTS), DB(MariaDB, 5명 정도 동시 접속), 응답 시간 1초 이내

지금이라면 어떻게 개선할지: p.20

인턴 엔지니어스톡 2023.07.03–08.25 · SW/AI사업단 2023 하계방학 인턴십 **총 315시간** 이수 · DSC 프로젝트(장려상)에서 다룬 고장 접수 챗봇 과제가 인턴으로 이어졌습니다(p.17).

지금이라면 LLM이 증상을 범주에 매핑하고, 확정은 담당자가 하게 만들겠습니다

당시 만든 방식과, 지금 다시 한다면 바꿀 방식을 나란히 놓았습니다. 오른쪽은 **구현하지 않은 개선 제안**입니다.

당시 만든 방식

제품군 버튼과 증상 체크로 접수
4명이 쓴 말뭉치 기반

↓

고객 보조 텍스트는 Issue 테이블에 원문 저장

↓

최종 입력은 업무 담당자가 확인

자연어 접수는 발표 전 구조를 바꾸며 뺐습니다. 근거: 구조 전환 p.17, 자동화 범위 결정 p.18

→

지금이라면 · 구현하지 않은 개선 제안

수리내역과 분류 체계를 지식으로 삼아 **LLM이 증상 문장을 범주에 매핑**하고, 유사 사례를 근거로 보여주되 **확정은 담당자가** 하는 구조로 개선하겠습니다.

증상 문장 입력

↓

LLM이 범주 매핑

↓

유사 수리 사례 제시

↓

담당자가 최종 확정

* 오른쪽 점선 영역은 구현하지 않은 개선 제안입니다.

큰 데이터를 옮기고 조사하기 전에, 순서와 기준부터 문서로 정했습니다

두 인턴 모두 **작업 순서와 검사 기준을 먼저 정하고**, 나눠서 작업한 결과를 맞춰 보며 끝까지 확인했습니다.

2023.01.02–02.24 · 충남정보문화산업진흥원 융합콘텐츠본부 인턴

SKT 데이터 약 6,000만 건 DB 이관과 검증

내 담당 기획안, DB 설계 계획 보고서 · ERD 정규화, CSV 병합 코드

60,874,819건

로컬 MySQL → NAS MariaDB 이관 (인턴 3명)

PC 3대

1대로 10시간 54분 → 병렬 이관

- **1/3–1/27 기획안:** 데이터 안심구역 사례, 지정 요건, 점검항목, 공공데이터 활용 사례, 사업성을 조사해 최종본을 제출했습니다.
- **DB 설계 계획 보고서:** 데이터 파악 → CSV 병합 → 표준화 → 오류값 정리 → ERD 설계 · 정규화 → NAS DB 구축 → 이관 → 검사 순서와, <null>과 빈 문자열 · 1과 01 · 문자열 숫자를 맞추는 오류값 기준을 먼저 정했습니다. ERD 설계와 정규화는 제가 맡았습니다.
- **1/30–2/13 CSV 병합:** SKT, 현대카드(2019~2022년), BC카드 데이터를 월별, 항목별로 합치는 코드를 짰습니다.
- **이관 검증:** 로컬 DB와 NAS DB의 테이블별 데이터 수와 속성 수를 하나씩 대조해 누락을 확인했습니다.

사용 도구 **MySQL, MariaDB, DataGrip, phpMyAdmin, ERD Cloud, Notion**

2022.09.26–12.16 · ㈜아이와즈 지능화기술연구팀 인턴

KISA 웹하드 개인정보 침해 현황 조사

내 담당 조사 · 크롤러 설계 · 보안 도구 분석, 학습 데이터 태깅

약 140만 건

크롤러로 조사한 웹하드 게시물

약 2만 건

그중 키워드 해당 게시물

- **원격 서버:** kt cloud 원격 서버에 작업 환경을 세팅했고, 서버가 멈추자 사수에게 알리고 다른 서버에 분석 프로그램을 다시 설치해 이어 갔습니다.
- **크롤러:** 처음 익힌 Selenium으로, 사이트마다 다른 페이지 이동 단위(1, 5, 10, 15페이지)에 맞춘 게시물 수집계 코드를 설계해 사수에게 구조를 설명했습니다.
- **진위 대조:** 웹하드 28곳의 투명성 보고서를 실제 사이트와 대조했습니다. URL 쿼리가 막힌 곳은 Burp Suite로 GET 파라미터를 바꿔 이동하고, 전용 프로그램은 x32dbg로 분석해 서버 도메인과 IP를 추적했습니다.
- **학습 데이터(KT 과제):** 한국기술교육대학교 도로노면 표시 상태 데이터 태깅, 11월 말부터 다른 인턴의 결과 검수

사용 도구 **Python, Selenium, Chrome Driver, Burp Suite, x32dbg, kt cloud**

그 밖의 프로젝트

YOLO 학습과 실패 분석, 현장 요구 재정의, 앱 개발 팀 리딩

직무와 연결되는 나머지 프로젝트 4건입니다. 잘 안 된 부분도 원인과 함께 적었습니다.

실 데이터 기반 어린이보호구역 기준 및 입지 추가 제안

2023.09–12 · 4인(3개 대학) · 모빌리티 캡스톤디자인 II

내 역할 **AI 파트: 데이터 수집과 YOLOv5 학습 (팀원 1명과 공동)**

AI허브 어린이 보행 영상을 아이와 어른으로 다시 라벨링해(labelImg, 팀원들과 1,000장) yolov5x를 학습했습니다. 라벨링에 쓰지 않은 블랙박스 영상에서 결과 영상 확인 기준 약 90%로 구분했고(mAP 아님), 얼굴 모자이크가 성능 한계였습니다. 어린이 비율 50% 이상은 잠재 보호구역, 등하교 시간대이거나 80% 이상이면 30km/h 제한 기준을 세웠습니다. 지도 웹은 팀원 담당입니다.

교통사고 다발 구역 자동 분석 시스템

2023.03–06 · 3인(2개 대학) · 모빌리티 캡스톤디자인 I

내 역할 **이미지 수집 코드, 라벨 변환, YOLOv4-tiny 학습**

사고 좌표로 스트리트뷰 4방향을 저장하는 코드를 짜고, AI허브 json 라벨을 Darknet용 txt(6개 클래스)로 바꿔 yolov4-tiny.conv.29에서 학습했습니다. 9,536장도 예상 학습 시간이 약 20시간이라 무료 Colab 런타임 종료에 걸렸고, 1,000회 반복 때 가중치가 저장되지 않아 이어 학습도 못 해 목표에 못 미쳤습니다. 대안으로 체크포인트 저장 확인 후 이어 학습하는 절차와 CUDA · cuDNN 자체 환경을 제시해 논문 형식으로 정리했습니다(학회 미발표).

AI, AR 키오스크 ‘마이포스터’ 이벤트 콘텐츠 기획

2022.07–08 · 3인 · 기업애로기술 해결프로젝트 랩 (더에스, 6회 30시간)

내 역할 **사용자 인터뷰, 콘텐츠 아이디어**

‘이벤트 콘텐츠 개발’이라는 추상적인 과제를 디자인씽킹으로 다시 정했습니다. 페르소나에 맞는 지인 3명을 직접 인터뷰해 “결혼식장에 사진 키오스크가 있으면 쓰겠다”, “축의금을 카드로 낼 수 있으면 편하겠다”는 답을 얻었고, 이를 근거로 하객이 축의금을 접수하고 포토카드를 출력하는 흐름을 기획해 기업 앞에서 발표했습니다.

Wear OS 취약자 보호 앱 ‘Safety Watcher’

2022.03–06 · 4인 · 캡스톤디자인

내 역할 **팀장(기술 선택), 앱 개발**

POC로 GPS, 생체 정보, 음성 녹음이 가능한지 공식 가이드로 확인하고, 팀장으로서 Kotlin과 (MVP 목표에 맞춰 AWS 대신) Firebase를 정했습니다. 로그인 · UI, 워치 GPS 수신, 보호자 앱 지도 마커는 각각 구현했지만 둘을 잇는 Firebase 연동과 워치 위치 권한 오류는 학기 안에 해결하지 못했습니다(결과보고서 기준 약 15%). 핵심 기능은 실제 기기에서 먼저 돌려 보고 범위를 정해야 한다는 것을 배웠습니다.

띄운 서비스가 안전하게 계속 돌아가도록,

이상은 먼저 알아채고 반복 작업은 스크립트로 옮기겠습니다.

클라우드 운영

AWS 구축, 배포 자동화, CORS 장애 추적, SSH 점검

Project 01, p.6–11

제조 현장 DX

조선소 디지털 트윈 범위 재설계, 15개 시설 GeoJSON 지도 구현

Project 01, p.12–13

업무 시스템 분석과 설계

고장 접수 현행 분석, 증상 체계, 구조 전환, 3갈래 개선 설계

Project 02, p.14–20

GitHub github.com/jjaehani Tel 010-4654-0406 Email jjaehani@kakao.com